MNR

Fachliche RAG-Architektur & Implementierungskonzept

Meta-Normen-Register | KI-Plattform für Normendigitalisierung

Version: 1.1
Stand: 16. April 2026
Rolle: Product Owner & Enterprise Architect | Public Sector / Digital Governance
Lizenz: Ausschließlich Open-Source-Technologien | On-Premise-fähig | BSI-konform

Inhaltsverzeichnis

1. Überblick & Architekturziele ... 3
2. Zweigeteilter Gesamtprozess ... 3
3. Architektur-Übersicht (Komponentensicht) ... 4
4. Phase 1: Ingest-Pipeline im Detail ... 4
   4.1 Dokumenten-Ingestion & Parsing ... 4
   4.2 Chunking-Strategie ... 5
   4.3 NLP: Subjekt-Objekt-Prädikat-Extraktion (SVO) ... 7
   4.4 Embedding & Vektorspeicherung ... 8
   4.5 Klassen-sensitives Retrieval ... 9
5. Phase 1: RAG-Engine & Informationsmodell-Generierung ... 10
   5.1 RAG-Architektur ... 10
   5.2 Prompt-Suite (Standard-Abfragen) ... 10
   5.3 Informationsmodell-Struktur (IM) ... 11
6. Knowledge Graph ... 12
7. Human-in-the-Middle (Fachaufsicht-Workflow) ... 12
8. Phase 2: Artefakt-Generierung ... 13
   8.1 Architektur-Agent ... 13
   8.2 Traceability in Artefakten ... 13
9. Vollständiger Technologie-Stack ... 14
   9.1 Core Infrastructure ... 14
   9.2 KI / ML Stack ... 14
   9.3 Frontend / Portal & Monitoring ... 15
10. Implementierungs-Roadmap ... 15
11. Qualitätssicherung & Evaluierung ... 16
   RAG-Qualitätsmetriken ... 16
   IM-Qualitätssicherung ... 16
12. Sicherheit & Datenschutz ... 16
13. Wiederverwendbarkeit & Skalierung ... 17
Anhang A: Entscheidungsmatrix Open-Source-Technologien ... 18
Anhang B: Architekturdiagramme ... 18
   B.1 Gesamtarchitektur MNR – Zweiphasiger Prozess ... 18
   B.2 Ingest-Pipeline mit Chunking-Router und dreiklassiger Strategie ... 19
Anhang C: Klassen-sensitiver Retrieval-Pfad ... 20
   C.1 Retrieval-Flussdiagramm ... 20

1. Überblick & Architekturziele

Das Meta-Normen-Register (MNR) erfordert eine fachagnostische, skalierbare KI-Plattform, die aus normativen Grundlagen (Gesetze, Verordnungen, Standards) automatisiert ein verifizierbares Informationsmodell erzeugt und daraus standardkonforme technische Artefakte ableitet. Die Architektur folgt dem Prinzip 'Law as a System Source' und bildet eine durchgaengige digitale Wertschöpfungskette von der Rechtsnorm bis zum deployten Register.

Architekturgrundsätze

Prinzip	Ausprägung
Open Source Only	Ausschließlich OSS-Komponenten; kein proprietäres Lock-in
Traceability	Jede Aussage im Modell ist auf eine Rechtsquelle zurückführbar
Human-in-the-Middle	Fachaufsicht genehmigt jeden Übergang zwischen Phasen
Fachagnostik	Plattform ist behörden- und registerunabhängig wiederverwendbar
Interoperabilität	XÖV, OpenAPI, JSON-LD, RDF/OWL als Ausgabeformate
Datensouveränität	On-Premise-fähig, vollständig selbst betreibbar

2. Zweigeteilter Gesamtprozess

Der Prozess ist streng in zwei entkoppelte Phasen gegliedert. Phase 1 kann mehrfach iteriert werden, bis das Informationsmodell durch die Fachaufsicht freigegeben ist. Erst dann wird Phase 2 ausgelöst.

Phase	Beschreibung	Gate-Bedingung
Phase 1 – Informationsmodell	Normative Quellen → RAG-Analyse → Informationsmodell (IM) → iterative Fachaufsicht-Freigabe (Human-in-the-Middle)	Freigabe durch Fachaufsicht
Phase 2 – Artefakt-Generierung	Bestätigtes IM → Architektur-Agent → Technische Artefakte (DB-Schema, OpenAPI, Validierungsregeln, Register-Blueprint)	Deployment durch Registerfactory

3. Architektur-Übersicht (Komponentensicht)

Die Plattform gliedert sich in sechs Schichten. Alle Schichten sind über standardisierte REST-APIs verbunden. Das MN-Pflege-Portal stellt die Governance- und Review-Oberfläche bereit. Detaillierte Komponentendiagramme finden sich in Anhang B.

Schicht	Komponente	Funktion
Presentation	MN-Pflege-Portal (Next.js)	Review-UI, Traceability-View, Freigabe-Workflow
API Gateway	FastAPI + Keycloak/OIDC	Routing, Auth, Rate-Limiting, Audit-Log
Processing	Ingest-Pipeline, RAG-Engine (LlamaIndex)	Dokumentenverarbeitung, Normenanalyse
AI/ML	Ollama/vLLM, Mistral 7B, spaCy, Flair	LLM-Inferenz, NLP, Embedding
Storage	PostgreSQL + pgvector, Apache Jena, MinIO	Chunks, Embeddings, Knowledge Graph, Artefakte
Orchestrierung	Apache Airflow	Pipeline-DAGs, Freigabe-Trigger, Monitoring

4. Phase 1: Ingest-Pipeline im Detail

4.1 Dokumenten-Ingestion & Parsing

Eingabeformate: PDF, HTML, DOC, DOCX, RTF

Schritt	Aufgabe	Technologie
Dokumenten-Empfang	Upload-API, Dateivalidierung, Versionierung	FastAPI + MinIO
Format-Parsing	Multi-Format-Extraktion (Text, Struktur, Tabellen)	Apache Tika 2.x (primär) + Docling
Struktur-Erkennung	Kapitel, Absätze, §§-Erkennung, Randnummern	Docling + Custom-Parser (Regex + spaCy)
Metadaten-Extraktion	Normtyp, Geltungsbereich, Datum, Jurisdiktion, Version	Regelbasiert + LLM-Klassifikation
Textnormierung	Zeichensatz, Bindestriche, Absätze, Abkürzungen	Custom Python-Pipeline

Metadaten-Schema je Dokument (JSON):

```
{
 "doc_id": "uuid",
 "source_type": "gesetz|verordnung|standard|leitfaden",
 "title": "Meldegesetz BW",
 "jurisdiction": "BW",
 "valid_from": "2024-01-01",
 "valid_to": null,
 "norm_reference": "§ 1 MeldeG BW",
 "version": "2.1",
 "language": "de",
 "register_scope": ["einwohnerregister"],
 "approved_by": "user_id",
 "ingest_ts": "2026-04-16T10:00:00Z"
}
```

4.2 Chunking-Strategie

Das MNR verarbeitet drei strukturell verschiedene Dokumentklassen, die unterschiedliche Chunking-Strategien erfordern. Voraussetzung ist die automatische Dokumenttyp-Erkennung beim Ingest, die anhand von Strukturmerkmalen (§-Marker, Heading-Hierarchie, Inhaltsverzeichnis) den Chunking-Router steuert.

Dokumenttyp-Erkennung (Chunking-Router)

Der Router prüft in einer Kaskade: zuerst §-Marker (→ Klasse A), dann Heading-Hierarchie (→ Klasse B), dann Fallback auf semantisches Chunking (→ Klasse C). Die erkannte Klasse wird als doc_class-Feld persistent im Chunk-Metadatum gespeichert.

Erkennungsmerkmal	Ergebnis	Chunking-Strategie
§-Marker / amtl. Normformat vorhanden	Klasse A	§-basiertes hierarchisches Chunking
Heading-Hierarchie (H1/H2/H3 oder 1.1.2)	Klasse B	Kapitel-basiertes hierarchisches Chunking
Kein stabiles Strukturgerüst	Klasse C	Semantisches Sliding-Window-Chunking

Klasse A: Normative Rechtstexte

Strukturprinzip: §-Paragraphen sind die primäre Sinneinheit. Die Granularität folgt der Rechtshierarchie Paragraph → Absatz → Satz.

Ebene	Token-Limit	Overlap	Besonderheit
Parent: §-Paragraph	max. 1.024	keiner	Gesamtparagraph als Kontext-Anker
Child: Absatz	max. 256	keiner	chunk_type: tatbestand | rechtsfolge | definition | ausnahme | zustaendigkeit
Child: Satz (optional)	max. 128	keiner	Nur bei sehr langen Absätzen

Ebene	Token-Limit	Overlap	Besonderheit
Legaldefinitionen	max. 256	keiner	Eigener Chunk unabhängig von Position; chunk_type=definition
Verweise	max. 128	keiner	chunk_type=verweis; cross_references-Feld gesetzt

Klasse B: Strukturierte Fach-Dokumente

Strukturprinzip: Das Inhaltsverzeichnis (ToC) ist die Strukturreferenz. Kapitel und Unterkapitel sind die primären Sinneinheiten – analog zur §-Ebene, aber mit inhaltlicher statt normativer Logik.

Ebene	Token-Limit	Overlap	Besonderheit
Parent: Kapitel	128 (Titel + Einleitung)	keiner	Kontextanker für alle Unterkapitel
Child: Unterkapitel	max. 512	15% zum Vorgänger	heading_breadcrumb im Metadatum
Tabellen-Chunk	variabel	keiner	JSON-kodiert; Rückreferenz auf Abschnitts-Chunk
Anforderungs-Mini-Chunk	max. 128	keiner	requirement_id (z. B. A-001) als Metadatum

Klasse C: Unstrukturierte Ergänzungstexte

FAQs, Handreichungen, Auslegungshinweise ohne stabiles Strukturgerüst. Fließen als Auslegungswissen in die Wissensbasis; confidence_weight = 0,65.

Parameter	Wert
Chunking-Methode	Sentence-Splitter mit semantischer Ähnlichkeit
Segment-Grenze	Cosine-Similarity < 0,65
Token-Bereich	min. 3 Sätze / max. 384 Token
Overlap	20% Sliding-Window
confidence_weight	0,65 (niedrigste Priorität)
source_type	auslegung

Gemeinsames Metadaten-Schema (alle Klassen)

```
{
 "chunk_id": "uuid",
 "doc_id": "uuid-ref",
 "doc_class": "A|B|C",
 "source_type": "gesetz|verordnung|standard|fachkonzept|leitfaden|lastenheft|auslegung",
 "norm_reference": "§ 3 Abs. 2 MeldeG BW",        // Klasse A
```

```
  "section_path": "3.2.1",                    // Klasse B
  "heading_breadcrumb": "Datenanf. > Opt. Felder > Datum", // Klasse B
  "requirement_id": "A-001",                  // Klasse B (Lastenhefte)
  "chunk_type": "tatbestand|rechtsfolge|definition|ausnahme|anforderung|tabelle|verweis",
  "hierarchy_level": 2,
  "parent_chunk_id": "uuid-ref",
  "cross_references": ["uuid-ref-2"],         // Klasse A
  "overlap_with_prev": 0.15,
  "confidence_weight": 1.0,                   // A=1.0 | B=0.85 | C=0.65
  "text": "...",
  "token_count": 128,
  "version": "2.1",
  "valid_from": "2024-01-01"
}
```

4.3 NLP: Subjekt-Objekt-Prädikat-Extraktion (SVO)

Die Normenanalyse extrahiert aus jedem Chunk die operativen Bestandteile: Subjekte (Akteure/Rollen), Objekte (Datenobjekte), Prädikate (Handlungen/Pflichten) sowie Normtypen.

Beispiel:

```
Normtext: "Die Meldebehörde speichert die Meldedaten der betroffenen Person."
→ Subjekt:   Meldebehörde      [Rolle/Akteur]
→ Prädikat:  speichert         [Pflicht/Handlung]
→ Objekt:    Meldedaten        [Datenobjekt]
→ Kontext:   betroffene Person  [Bezugsobjekt/Rolle]
→ Normtyp:   Pflicht (§ X ...)  [Regeltyp: MUST]
```

NLP-Schritt	Technologie	Modell
Tokenisierung + POS-Tagging	spaCy 3.x	de_core_news_lg
Named Entity Recognition	spaCy + Flair	de-ner-legal (custom fine-tuned)
Dependency Parsing (SVO)	spaCy	de_core_news_lg
Normtyp-Klassifikation	Hugging Face Transformers	deepset/gbert-large (fine-tuned)
Semantische Ähnlichkeit	Sentence-Transformers	paraphrase-multilingual-mpnet-base-v2
Koreference Resolution	spaCy coreferee	Deutsch-Modell

Normtyp	Beispielformulierung	Tag
Pflicht	muss / ist zu / hat ... zu	MUST
Ermessen	kann / darf / ist berechtigt	MAY

Normtyp	Beispielformulierung	Tag
Verbot	darf nicht / ist untersagt	MUST_NOT
Definition	im Sinne dieses Gesetzes ist	DEF
Ausnahme	es sei denn / sofern nicht	EXCEPT
Frist	binnen / spätestens / unverzüglich	DEADLINE
Zuständigkeit	zuständig ist / obliegt	COMPETENCE

4.4 Embedding & Vektorspeicherung

Embedding-Modell (primär): multilingual-e5-large (Microsoft, Apache 2.0), Dimension 1024, spezialisiert auf mehrsprachige semantische Suche. Fallback für rein deutsche Texte: deepset/gbert-large.

pgvector-Schema (DDL):

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE norm_chunks (
  id                UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  doc_id            UUID NOT NULL REFERENCES norm_documents(id),
  doc_class         CHAR(1) NOT NULL CHECK (doc_class IN ('A','B','C')),
  norm_reference    TEXT,
  cross_references  UUID[],
  section_path      TEXT,
  heading_breadcrumb  TEXT,
  requirement_id    TEXT,
  chunk_type        TEXT,
  hierarchy_level   INT,
  parent_id         UUID REFERENCES norm_chunks(id),
  overlap_with_prev  NUMERIC(4,2),
  confidence_weight  NUMERIC(3,2) NOT NULL DEFAULT 1.0,
  content           TEXT NOT NULL,
  content_hash      TEXT GENERATED ALWAYS AS (md5(content)) STORED,
  entities          JSONB,
  metadata          JSONB,
  embedding         VECTOR(1024),
  version           TEXT,
  valid_from        DATE,
  valid_to          DATE,
  created_at        TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON norm_chunks USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
CREATE INDEX ON norm_chunks USING gin (metadata);
CREATE INDEX ON norm_chunks (doc_id, doc_class);
CREATE INDEX ON norm_chunks (norm_reference)  WHERE norm_reference  IS NOT NULL;
CREATE INDEX ON norm_chunks (section_path)    WHERE section_path    IS NOT NULL;
CREATE INDEX ON norm_chunks (requirement_id)  WHERE requirement_id  IS NOT NULL;
CREATE INDEX ON norm_chunks USING gin (cross_references) WHERE cross_references IS NOT NULL;
```

4.5 Klassen-sensitives Retrieval

Die drei Dokumentklassen erfordern nicht nur unterschiedliches Chunking, sondern auch unterschiedliche Retrieval-Modi. Die Unterschiede betreffen vier Dimensionen.

Dimension	Klasse A – Rechtstext	Klasse B – Fach-Dok.	Klasse C – Ergänzung
Direkt-Lookup	Ja, über norm_reference	Ja, über requirement_id	Nein
Semantischer Retriever	Vektor-Cosine + FTS	Vektor-Cosine + FTS	Nur Vektor-Cosine
Score-Threshold	≥ 0,60	≥ 0,60	≥ 0,72 (erhöht)
Parent-Fetch	Immer (§-Parent)	Selektiv (bei Abschneidung)	Nein
Post-Retrieval-Expansion	Cross-Reference-Auflösung	Tabellen-Chunk → Abschnitt	Keine
Kontext-Anreicherung	Normreferenz	heading_breadcrumb	Keine
confidence_weight	1,0	0,85	0,65
Priorisierung	Höchste	Mittel	Nur als Supplement

Dimension 1: Query-Routing

Liegt eine bekannte Normreferenz in der Anfrage vor, wird für Klasse A ein direkter Metadaten-Lookup auf das Feld norm_reference durchgeführt – kein Vektorvergleich. Analog kann bei Klasse B eine requirement_id als direkter Schlüssel dienen. Klasse C hat keine kanonischen Identifikatoren und geht immer über den semantischen Pfad.

Dimension 2: Parent-Child-Verhalten

Bei Klasse A trifft der Retriever typischerweise einen Child-Chunk (Absatz-Ebene). Der Parent-Chunk (gesamter §) wird automatisch nachgeladen, weil Absätze ohne den Paragraphenkontext oft nicht selbstständig interpretierbar sind. Bei Klasse B ist das Parent-Nachladeverhalten selektiver: der Kapitel-Parent wird nur dann geladen, wenn der Unterkapitel-Chunk erkennbar am Token-Limit abgeschnitten wurde. Tabellen-Chunks lösen immer einen Fetch ihres beschreibenden Abschnitts aus.

Dimension 3: Cross-Reference-Expansion (nur Klasse A)

Klasse A ist die einzige Quellenklasse mit expliziten cross_references-Feldern. Nach dem initialen Retrieval werden referenzierte Chunks automatisch nachgeladen. Das ist bei Rechtstexten unverzichtbar: eine Norm wie § 5 gilt entsprechend ist ohne den Inhalt des § 5 nicht auswertbar.

Dimension 4: Confidence-Gewichtung & Score-Threshold

Der erhöhte Score-Threshold für Klasse C (≥ 0,72 statt ≥ 0,60) wirkt als Qualitätsfilter: da Handreichungen semantisch breiter formuliert sind, produzieren sie bei schwachen Queries

trotzdem mittelmäßige Scores. Ohne diesen Filter würden Klasse-C-Dokumente systematisch präzisere Klasse-A-Chunks verdrängen.

Duplikat-Entfernung & Klassen-Kennzeichnung im Prompt

Der Re-Ranker führt eine Duplikat-Entfernung auf Basis des content_hash durch (relevant besonders für Klasse B wegen 15%-Overlap). Das LLM erhält die Klassenkennung explizit als Teil des Prompts, damit es in der Traceability-Ausgabe zwischen verbindlicher Rechtsnorm (Klasse A) und fachlicher Auslegung (Klasse C) unterscheiden kann.

5. Phase 1: RAG-Engine & Informationsmodell-Generierung

5.1 RAG-Architektur

Framework: LlamaIndex (primär) + Haystack (für Pipelines).

Stufe	Funktion	Technologie
Query-Transformation	HyDE, Step-Back, Multi-Query	LlamaIndex
Hybrid Retriever	Semantic (pgvector cosine) + Lexikalisch (PostgreSQL FTS)	LlamaIndex + pgvector
Re-Ranker	Klassen-gewichtetes Scoring, Duplikat-Entfernung	cross-encoder/ms-marco-MiniLM
Context Assembly	Parent-Child Merge, KG-Lookup, Klassen-Kennzeichnung	LlamaIndex
LLM	Mistral 7B Instruct / Llama 3.1 8B, JSON-Mode	Ollama / vLLM
Output Validation	Pydantic v2 Schema-Validierung, Traceability-Mapper	Pydantic v2

5.2 Prompt-Suite (Standard-Abfragen)

Die Prompt-Suite ist die wiederholbare, normierte Abfragemenge, die für jedes Register ausgeführt wird:

Prompt-ID	Ziel	Output-Schema
PS-01	Entitäten & Datenobjekte extrahieren	EntityList
PS-02	Attribute & Datentypen je Entität	AttributeMap
PS-03	Beziehungen & Kardinalitäten	RelationGraph
PS-04	Rollen & Zuständigkeiten	RoleMatrix
PS-05	Pflichten, Verbote, Ermessen	NormRuleSet

Prompt-ID	Ziel	Output-Schema
PS-06	Fristen & Prozessregeln	ProcessConstraints
PS-07	Ausnahmen & Sonderfälle	ExceptionList
PS-08	Definitionen & Begriffsklärungen	Glossary
PS-09	Offene Punkte & Interpretationsfragen	OpenIssues
PS-10	Quellen-Traceability-Matrix	TraceabilityMap

5.3 Informationsmodell-Struktur (IM)

Das Informationsmodell ist das Kernprodukt von Phase 1. Es ist technologieneutral und prüfbar durch die Fachaufsicht. Struktur (YAML-Beispiel: Einwohnermelderegister):

```yaml
im_version: "1.0-draft"
register_name: "Einwohnermelderegister BW"
status: "draft"   # draft | review | approved | rejected

entities:
  - id: "ENT-001"
    name: "Person"
    source_ref: "§ 1 Abs. 1 MeldeG BW"
    norm_type: "MUST"
    attributes:
      - id: "ATTR-001"
        name: "familienname"
        data_type: "string"
        required: true
        source_ref: "§ 3 Abs. 1 Nr. 1 MeldeG BW"
        constraints: ["max_length: 200", "not_null"]

rules:
  - id: "RULE-001"
    type: "DEADLINE"
    description: "Anmeldung binnen zwei Wochen nach Einzug"
    expression: "anmeldedatum <= einzugsdatum + 14 Tage"
    source_ref: "§ 17 Abs. 1 MeldeG BW"

traceability:
  - claim_id: "ATTR-001"
    citation: "Als Grunddaten sind zu speichern: ... Familienname..."
    doc_id: "uuid-1"
    chunk_id: "chunk-uuid-42"
    norm_reference: "§ 3 Abs. 1 Nr. 1 MeldeG BW"
```

6. Knowledge Graph

Der Knowledge Graph ergänzt die Vektordatenbank um explizite semantische Relationen auf Basis von RDF/OWL. Er ermöglicht Impact-Analysen bei Gesetzesänderungen, modelliert explizite Abhängigkeiten zwischen Registern und unterstützt kontextsensitives Retrieval (Graph-RAG).

Aspekt	Ausprägung
Technologie	Apache Jena Fuseki / Oxigraph (leichtgewichtig)
Abfragesprache	SPARQL 1.1
Modellierungssprache	RDF / OWL
Ontologie-Basis	FOAF + Verwaltungsontologie (LIDO/XÖV)
Einsatz 1	Impact-Analyse bei Gesetzesänderungen
Einsatz 2	Explizite Abhängigkeiten zwischen Registern
Einsatz 3	Kontextualisiertes Retrieval (Graph-RAG)

7. Human-in-the-Middle (Fachaufsicht-Workflow)

Die Fachaufsicht ist technisch erzwungen, nicht optional. Jeder IM-Draft muss explizit freigegeben werden, bevor Phase 2 ausgelöst wird.

Schritt	Aktion	Ergebnis
1	Agent erzeugt IM-Draft aus RAG-Ergebnis	IM v1 (draft)
2	Fachaufsicht prüft im MN-Pflege-Portal: Vollständigkeit, Korrektheit, Offene Punkte	Review-Entscheidung
3a	Freigabe	IM-Status → approved; Übergabe an Phase 2
3b	Ablehnung / Korrektur	Ergänzungsdokumente erzeugen → Wissensbasis → neue Iteration

Komponente	Technologie
Workflow-Engine	Apache Airflow DAG (Freigabe-Trigger) + FastAPI Webhook

Komponente	Technologie
Audit-Trail	PostgreSQL Tabelle im_review_log (Zeitstempel, User-ID, Aktion, Kommentar)
Versionierung	Jeder IM-Stand als versioniertes YAML in MinIO archiviert

8. Phase 2: Artefakt-Generierung

8.1 Architektur-Agent

Nach Freigabe des IM übernimmt der Architektur-Agent die technische Ableitung:

Artefakt	Format	Beschreibung
Datenbankschema	SQL (PostgreSQL)	DDL mit Constraints, Indizes, Fremdschlüsseln
Register-Blueprint	YAML (Factory-DSL)	Metadaten-Schema für Registerfactory
OpenAPI-Spezifikation	JSON/YAML (OpenAPI 3.1)	REST-Schnittstellen für Register-CRUD
Validierungsregeln	JSON / DMN 1.3	Fachliche Constraints aus Normen
XÖV-Datenstruktur	XML Schema (XSD)	Für XÖV-konforme Datenübertragung
Rollenmodell	JSON (RBAC)	Zuständigkeiten aus der Normenanalyse
Event-Definitionen	AsyncAPI / CloudEvents	Für ereignisgetriebene Registerarchitektur

8.2 Traceability in Artefakten

Jedes generierte Artefakt trägt Quellenverweise bis auf die Normreferenz zurück:

```
{
 "table": "person",
 "column": "familienname",
 "type": "VARCHAR(200)",
 "nullable": false,
 "_sourceRef": {
  "norm": "§ 3 Abs. 1 Nr. 1 MeldeG BW",
  "doc_id": "uuid-1",
  "chunk_id": "chunk-uuid-42",
  "im_claim": "ATTR-001"
 }
}
```

9. Vollständiger Technologie-Stack

9.1 Core Infrastructure

Schicht	Komponente	Version / Lizenz
Datenbank	PostgreSQL + pgvector	16+ / Apache 2.0
Objektspeicher	MinIO	AGPL 3.0
Message Queue	Apache Kafka	Apache 2.0
Orchestrierung	Apache Airflow	Apache 2.0
Identität / AuthN	Keycloak (OIDC/OAuth2)	Apache 2.0
Containerisierung	Docker + Kubernetes (K3s)	Apache 2.0
API-Gateway	FastAPI + Nginx	MIT / BSD

9.2 KI / ML Stack

Komponente	Technologie	Modell / Variante
LLM-Serving	Ollama (Dev) / vLLM (Prod)	Mistral 7B Instruct v0.3
LLM (Reasoning)	vLLM	Llama 3.1 8B Instruct
Embedding	Sentence-Transformers via FastEmbed	multilingual-e5-large
NLP Pipeline	spaCy 3.x	de_core_news_lg
NER (Recht)	Flair + spaCy	Custom fine-tuned auf Verwaltungsrecht
Klassifikation	Hugging Face Transformers	deepset/gbert-large
RAG Framework	LlamaIndex 0.10+	–
Pipeline Framework	Haystack 2.x	–
Re-Ranker	Cross-Encoder	ms-marco-MiniLM-L-12-v2
KG-Speicher	Apache Jena Fuseki	Apache 2.0
LLM Observability	Langfuse	MIT

9.3 Frontend / Portal & Monitoring

Bereich	Komponente	Technologie
MN-Pflege-Portal	Frontend	Next.js 14 + React
MN-Pflege-Portal	UI-Komponenten	shadcn/ui + Tailwind CSS
MN-Pflege-Portal	API-Client	TanStack Query
MN-Pflege-Portal	Visualisierung	React Flow / Mermaid.js
Monitoring	Metriken	Prometheus + Grafana
Monitoring	Logging	Loki + Grafana
Monitoring	Tracing	OpenTelemetry + Jaeger

10. Implementierungs-Roadmap

Meilenstein	Zeitraum	Ziel	Wesentliche Aufgaben
M1 – Ingest & Vector DB	Wo. 1–6	Dokumente einlesen, chunken, embedden, speichern	pgvector-Schema, Tika/Docling, Chunking-Pipeline, Embedding, Airflow DAG
M2 – NLP & SVO	Wo. 5–10	SVO und Normtypen aus normativen Texten extrahieren	spaCy-Pipeline, GBERT-Normtyp-Klassifikator, Flair NER, Knowledge Graph
M3 – RAG-Engine	Wo. 9–16	Vollständiger RAG-Zyklus für IM-Generierung	LlamaIndex, Hybrid-Retriever, Re-Ranker, Prompt-Suite PS-01–10, Langfuse
M4 – IM & Human-in-the-Loop	Wo. 14–20	Vollständiger Phase-1-Prozess mit Freigabe-Workflow	IM-Generator-Agent, Keycloak, MN-Pflege-Portal MVP, Audit-Log, Versionierung
M5 – Artefakt-Generierung	Wo. 19–26	Technische Artefakte aus bestätigtem IM generieren	Architektur-Agent, DDL/OpenAPI/DMN/XÖV-Generator, Artefakt-Store, FV-Agent
M6 – Produktionshärtung	ab Wo. 24	Plattform produktionsreif, behördenübergreifend betreibbar	K3s, Multi-Tenancy, EUR-Lex Connector, BSI-Härtung, Schulungsunterlagen

11. Qualitätssicherung & Evaluierung

RAG-Qualitätsmetriken

Metrik	Beschreibung	Zielwert
Retrieval Recall@K	Werden relevante Chunks gefunden?	> 0,90
Answer Faithfulness	Ist die Antwort durch Quellen belegt?	> 0,95
Context Precision	Sind alle retriegelten Chunks relevant?	> 0,80
Traceability Coverage	% der IM-Claims mit Normreferenz	100%
Halluzinationsrate	% unbelegt behauptete Fakten	< 0,02

Evaluierungsframework: RAGAS (Open Source, Python) + manuelle Annotation durch Juristen.

IM-Qualitätssicherung

- Vollständigkeitsprüfung: Alle PS-Prompt-Ergebnisse (PS-01–PS-10) vorhanden?
- Konsistenzprüfung: Keine widersprüchlichen Regeln oder Typen?
- Normreferenz-Validierung: Jede Aussage mit gültigem norm_reference oder section_path?
- Fachaufsicht-Review: Stichproben-Prüfung durch Juristen/Fachexperten

12. Sicherheit & Datenschutz

Anforderung	Maßnahme
Zugriffskontrolle	Keycloak RBAC (Analyst, Fachaufsicht, Deployer, Admin)
Datenverschlüsselung	TLS 1.3 in Transit; AES-256 at Rest (MinIO)
Audit-Trail	Unveränderliche Log-Tabelle mit User-ID + Zeitstempel
Mandantentrennung	Row-Level Security in PostgreSQL je Behörde
BSI Grundschutz	Härtung nach BSI IT-Grundschutz (OPS.1.1.3)
DSGVO	Kein Training auf personenbezogenen Daten; LLMs lokal
Keine Cloud-APIs	Alle LLMs On-Premise (Ollama/vLLM); kein OpenAI / kein Drittanbieter

13. Wiederverwendbarkeit & Skalierung

Das System ist von Grund auf fachagnostisch und mandantenfähig konzipiert:

- Template-basierte Prompt-Suite: Neues Register = neues Quelldokument + gleiche PS-01–PS-10 Pipeline
- Pluggable Parsers: Neue Dokumentformate über Tika-Plugin-Architektur
- Schema-Registry: IM-Schemas und Factory-DSLs in einer zentralen Registry (MinIO)
- Horizontal skalierbar: Ingest-Pipeline als Microservice auf K8s; LLM-Serving über vLLM mit GPU-Sharding
- Föderales Betriebsmodell: Jede Behörde betreibt eigene Mandanten-Instanz; gemeinsamer Normen-Pool über zentralen Knowledge Graph (Bund)

Anhang A: Entscheidungsmatrix Open-Source-Technologien

Kategorie	Gewählte Technologie	Begründung	Alternative
LLM	Mistral 7B / Llama 3.1	Beste DE-Performance, Apache 2.0	Phi-3, Qwen2
Embedding	multilingual-e5-large	Multilingual, hohe Qualität	GBERT-large
RAG Framework	LlamaIndex	Mächtiger als LangChain für komplexe RAG	Haystack
Vector DB	pgvector	Gesetzt; eliminiert separaten Vektorspeicher	Weaviate, Qdrant
Knowledge Graph	Apache Jena	Etabliert, SPARQL 1.1, RDF/OWL	Oxigraph
Parsing	Docling + Tika	Docling: beste PDF-Strukturerkennung	Unstructured.io
Orchestrierung	Airflow	Weit verbreitet, gut integrierbar	Prefect, Dagster
Portal	Next.js + shadcn	Produktionsreif, große Community	Nuxt, SvelteKit
Auth	Keycloak	BSI-empfohlen, OIDC/SAML	Authentik
LLM Observability	Langfuse	Einziger OSS-Stack für LLM-Tracing	Phoenix (Arize)

Anhang B: Architekturdiagramme

B.1 Gesamtarchitektur MNR – Zweiphasiger Prozess

Das Diagramm zeigt die vollständige Komponentenarchitektur des MNR in zwei vertikal getrennten Zonen. Phase 1 (oben, teal) enthält die Ingest-Pipeline mit Chunking-Router, den dreiklassigen Verarbeitungspfad (Klasse A/B/C), die NLP-/Embedding-Pipeline, den pgvector-Store mit Knowledge Graph, die RAG-Engine mit LLM-Backend sowie den Human-in-the-Middle-Freigabe-Workflow. Phase 2 (unten, blau) zeigt den Architektur-Agenten, den optionalen Fachverfahrens-Agenten und die fünf Artefakt-Typen, die von der Registerfactory weiterverarbeitet werden.

Die gestrichelte horizontale Linie zwischen beiden Phasen markiert den Gate-Punkt: nur ein durch die Fachaufsicht freigegebenes Informationsmodell löst Phase 2 aus.

Farb-Kodierung der Komponenten:

Farbe	Komponentengruppe
Teal	Datenspeicher und Infrastruktur (pgvector, MinIO)
Blau	Fachliche Eingangsdaten (Quelldokumente) und Artefakte
Lila	KI- und Verarbeitungskomponenten (RAG-Engine, Agenten)
Amber	LLM-Backend und externe Agenten
Grün	Ausgaben (Informationsmodell, freigegebene Artefakte)
Coral	Governance-Komponenten (Knowledge Graph, Human-in-the-Loop)

Traceability-Kette (Fußzeile des Diagramms): Jedes technische Artefakt in Phase 2 trägt einen _sourceRef-Verweis, der über Claim → Chunk → doc_class (A, B oder C) → norm_reference bzw. section_path bis zur originalen Rechtsquelle oder dem Fach-Dokument nachverfolgbar ist.

Die gestrichelte Rückkopplungslinie rechts im Phase-1-Bereich stellt den Iterationspfad dar: die Fachaufsicht lehnt den IM-Draft ab, erzeugt Ergänzungsdokumente, die in die Wissensbasis zurückfließen, und löst eine neue Analyse-Iteration aus.

B.2 Ingest-Pipeline mit Chunking-Router und dreiklassiger Strategie

Das Diagramm vertieft die Ingest-Pipeline aus B.1. Im oberen Teil sind die drei Eingangsklassen als farbcodierte Boxen dargestellt, ergänzt durch eine vierte Box für die unterstützten Eingabeformate mit dem Parsing-Stack (Apache Tika + Docling). Alle Eingaben laufen in den Chunking-Router zusammen, der anhand von Strukturmerkmalen die Dokumentklasse erkennt und die passende Strategie aktiviert.

Der Router ist eine regelbasierte Prüfkaskade: zuerst §-Marker (eindeutig für Klasse A), dann Heading-Hierarchie (Klasse B), dann Fallback auf semantisches Chunking (Klasse C). Die Klasse wird als doc_class-Feld im Chunk-Metadatum persistent gespeichert und steuert von diesem Punkt an alle nachgelagerten Prozesse – insbesondere das Retrieval-Verhalten (vgl. Anhang C).

Wesentliche Parameter je Strategie:

Strategie	Token-Limit Parent	Token-Limit Child	Overlap	Besonderheit
Klasse A – §-Chunking	max. 1.024	max. 256	Kein Overlap	Cross-references, Legaldefinitionen als eigene Chunks
Klasse B – Kapitel-Chunking	128 (Titel+Einl.)	max. 512	15% (Unterkapitel)	heading_breadcrumb, Tabellen-Chunks, requirement_id

Strategie	Token-Limit Parent	Token-Limit Child	Overlap	Besonderheit
Klasse C – Semantic-Chunking	–	max. 384	20% Sliding-Window	confidence_weight=0,65; Cosine-Threshold 0,65 für Grenzen

Anhang C: Klassen-sensitiver Retrieval-Pfad

C.1 Retrieval-Flussdiagramm

Das Diagramm zeigt den vollständigen Retrieval-Pfad einer Prompt-Suite-Anfrage von der Query-Analyse bis zum assemblierten LLM-Kontext. Nach einer gemeinsamen Query-Analyse- und Metadata-Pre-Filter-Stufe verzweigt der Pfad in drei parallele Spuren, eine je Dokumentklasse. Jede Spur zeigt zwei Verarbeitungsschritte: den Retrieval-Modus (inkl. Direkt-Lookup-Logik) und die Post-Retrieval-Expansion. Die drei Spuren konvergieren im Re-Ranker, der klassengewichtetes Scoring und Duplikat-Entfernung durchführt, bevor der Context Assembler den finalen LLM-Prompt aufbaut.

Erläuterung der drei Spuren

Klasse A (Rechtstext): Als einzige Klasse mit kanonischen Normreferenzen ermöglicht Klasse A einen Direkt-Lookup, der den Vektorspeicher vollständig umgeht. Die Cross-Reference-Expansion ist exklusiv für Klasse A: Verweisstrukturen werden automatisch aufgelöst, weil Normen ohne ihre Quernormen nicht vollständig interpretierbar sind. Der erhöhte Score-Threshold gilt hier nicht.

Klasse B (Fach-Dokument): Der Direkt-Lookup über requirement_id ist nur bei Lastenheft-Anforderungen möglich. Tabellen-Chunks lösen immer einen Fetch ihres beschreibenden Abschnitts aus. Der heading_breadcrumb wird dem Chunk als Kontext-Präambel angefügt, damit das LLM die Einordnung im Dokument versteht.

Klasse C (Ergänzungstext): Kein Direkt-Lookup, kein Parent-Fetch, keine Cross-Reference-Expansion. Der höhere Score-Threshold (≥ 0,72) wirkt als Qualitätsfilter. Klasse-C-Chunks erreichen den LLM-Kontext nur als Supplement.

Re-Ranker und Context Assembly

Die Duplikat-Entfernung über content_hash ist besonders für Klasse B relevant, wo der 15%-Overlap zu fast identischen Chunks in benachbarten Abschnitten führen kann. Das klassengewichtete Scoring stellt sicher, dass im assemblierten Kontext Klasse-A-Quellen vor Klasse-B- und diese vor Klasse-C-Quellen stehen.

Das LLM erhält die Klassenkennung explizit als Teil des Prompts. Damit kann es in der Traceability-Ausgabe zwischen verbindlicher Rechtsnorm (Klasse A · §-Referenz) und fachlicher Auslegung (Klasse C · FAQ-Handreichung) unterscheiden – eine für die Fachaufsicht im Review-Portal kritische Information zur Einschätzung der Belastbarkeit jedes Informationsmodell-Claims.

Technologieentscheidungen berücksichtigen die Anforderungen an Datensouveränität, BSI-Konformität und behördenübergreifende Wiederverwendbarkeit.